Copyrights @ HeyVenus Integrated Healthscience 2026

Technical Requirements Document: Nudge Planning & Delivery System

1. Overview

This document specifies the architecture, data model, APIs, and operational requirements for migrating nudge scheduling and delivery from an in-process Docker daemon to a queue-driven worker architecture on ECS Fargate. The backend remains a request/response API service; all background planning and delivery moves to independently scalable workers coordinated through PostgreSQL, Amazon SQS, and Amazon EventBridge.

2. Background and problem statement

The current implementation runs a scheduler daemon inside the same container as the web backend. Every EC2 instance behind the ALB therefore runs its own copy of the scheduler, which creates three problems:

1. **Duplicate execution risk.** N instances means N schedulers competing to send the same nudges, with no coordination mechanism.
2. **Coupled scaling.** Background throughput is tied to web instance count; we cannot scale delivery capacity without scaling the API tier (and vice versa).
3. **Deployment fragility.** Rolling deploys and instance failures interrupt in-flight scheduling work with no recovery semantics.

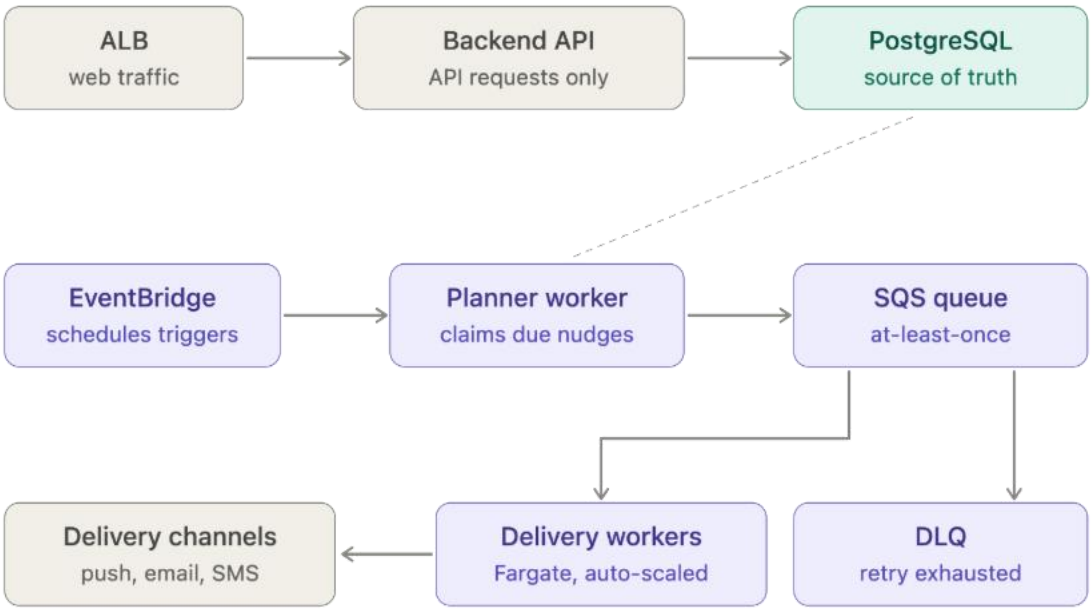
3. Goals

- G1: Exactly-once *effective* delivery per nudge occurrence (at-least-once transport with idempotent processing).
- G2: Delivery workers scale independently of the API tier, driven by queue depth.
- G3: Hour-level delivery-time precision (delivery on the hour), including per-user local-time delivery windows. The sweep cadence is a config value, so precision can be tightened later (e.g. to 15 or 1 minute) without architectural change.
- G4: Automatic retries for transient failures; dead-lettering and alerting for exhausted retries.
- G5: No new repository or standalone microservice; all services build from the existing codebase with separate entrypoints.
- G6: Zero duplicate sends during deploys, scale events, instance failure, or EventBridge double-fires.
- G7: Lean infrastructure cost. The pipeline targets < $15/month incremental spend at current volume and < $30/month at 10× volume (excluding channel-provider fees). Cost-minimizing defaults (scale-to-zero, Graviton, Spot burst capacity, no new NAT) are requirements, not optimizations to be added later.

4. Non-goals

- Sub-minute delivery precision.
- A general-purpose job/workflow engine (this system is scoped to nudges).
- Real-time (user-triggered, synchronous) notifications — those remain in the API request path.
- Multi-region active-active delivery (single region with DR posture for now).

5. High-level architecture



5.1 Components

Component	Runtime	Responsibility
Backend API	Existing EC2/ALB tier	CRUD for nudges, user preferences, occurrence history; never sends nudges
Planner worker	ECS Fargate (scheduled task or 1-task service)	Sweeps Postgres for due occurrences, claims them transactionally, enqueues to SQS
Delivery worker	ECS Fargate service (auto-scaled)	Consumes SQS, performs idempotent state transition, sends via channel provider, records outcome
PostgreSQL	Existing RDS	Source of truth for config, templates, occurrences, state

Component	Runtime	Responsibility
		machine, events, attempts
SQS main queue	Standard queue	At-least-once work distribution to delivery workers
SQS DLQ	Standard queue	Terminal parking for occurrences that exhausted transport retries
EventBridge	Scheduler rule	Fires the planner on a fixed cadence (hourly, aligned to the top of the hour)

5.2 Key design decisions

D1 — Postgres owns correctness; SQS owns distribution. SQS standard queues are at-least-once and unordered. All exactly-once semantics are enforced by a row-level state machine in Postgres with conditional transitions. FIFO queues are explicitly rejected: they cap throughput per message group and still cannot protect against a worker crashing after send but before delete.

D2 — Sweep-based scheduling, not per-nudge EventBridge schedules. A single EventBridge cron rule fires the planner hourly, aligned to the top of the hour. The planner queries for occurrences due in the current window. This keeps all schedule state in one queryable store and avoids reconciling millions of EventBridge schedule objects with the database. The sweep cadence *is* the delivery precision: because sweeps run at :00 and schedules are validated to be hour-aligned (§10.1), nudges are delivered on time. Tightening precision later is a config change to the cron expression plus relaxing the validation rule — no architectural change (G3).

D3 — Occurrences are materialized rows. The planner expands the programme (template `cadence_day` vs each user's day number) into concrete `nudge_occurrence` rows ahead of delivery. Materialized rows give trivial dedup via a unique index on `(user_id, template_id, occurrence_date_local)`, make "what will be sent and when" directly queryable by the API, and give every producer (routine, doctor follow-ups, JIT) one table to write into and one pipeline to drain.

D4 — Modular monolith deployment. One repository, one container image, three entrypoints (`api`, `planner`, `worker`), three ECS task definitions. Independent deploys and scaling without a microservice split (G5).

6. Data model

Integration principle: **reuse the reply/analytics spine untouched, replace the scheduling layer, promote the template table to authority.** The reply path (`nudge_event` → `nudge_response` → `metric_observation` / `chat_memories`) works and has downstream consumers; nothing in this design alters those tables. What gets replaced is the scheduling and idempotency layer — the daemon's tick logic, the Python cadence/registry, and `nudge_delivery_log` — with a state-machine occurrence table that

any producer (routine programme, doctor follow-ups, JIT) can write into and a single delivery pipeline drains.

6.1 Disposition of existing tables

Existing table / component	Disposition
`nudge_schedule_config`	**Reused** as planner input, with alterations (§6.3): local-time preference, hour alignment, `max_nudges_per_day` actually enforced, `enabled` authoritative
`nudge_template`	**Reused and promoted** to the single content authority (§6.4); Python registry becomes seed migrations. Closes the registry/table split — the biggest structural inconsistency in the current system
`nudge_event`	**Kept unchanged** — immutable send log, written by the delivery worker on send. `external_message_id` finally populated (§9.1), fixing reply attribution
`nudge_response`	Kept unchanged; entire inbound reply path untouched
`metric_observation`, `chat_memories`	Kept; the existing write logic moves verbatim from the daemon into the delivery worker
`user_profile`, `users`, `user_migration_map`	Read once at materialization; occurrences store the resolved `users.user_id` so the hot path never touches the migration map
`nudge_delivery_log`	**Deprecated** (already slated). Its idempotency-guard role is replaced by the occurrence state machine + unique constraint. Writes stop at cutover; dropped 90 days after Phase 6 (§16)
`dep_nudge_template`	Dropped in Phase 0
`user_metric_baseline`, `response_tags`	Declared but unwritten today; out of scope — implement-or-drop decision tracked in §18
`jit_schedules`	Kept as-is; becomes an occurrence *producer* in Phase 4 (§16) instead of a separate delivery path
`pulse_responses`, `scheduler_prompts`	Untouched
`nudge_scheduler.py` (standalone CLI)	Dead code (nothing imports it); deleted in Phase 3

Copyrights @ HeyVenus Integrated Healthscience 2026

6.2 New tables

Two new tables. All timestamps `timestamptz` in UTC; IDs are UUIDv7. Names are singular, matching house style (`nudge_event`, `nudge_response`).

```
CREATE TABLE nudge_occurrence (
    id                    uuid PRIMARY KEY,
    user_id               uuid NOT NULL REFERENCES users(user_id),
    template_id           uuid REFERENCES nudge_template(template_id),
    source                text NOT NULL DEFAULT 'routine'
                          CHECK (source IN
('routine','doctor_followup','jit')),
    channel               text NOT NULL DEFAULT 'whatsapp'
                          CHECK (channel IN ('whatsapp','sms')),
    occurrence_at         timestamptz NOT NULL,      -- resolved UTC delivery
time
    occurrence_date_local date NOT NULL,             -- computed in the
USER's tz at
                                                     -- materialization (not
hardcoded SG)
    status                text NOT NULL DEFAULT 'pending'
                          CHECK (status IN
('pending','enqueued','sending','sent',
                                              'failed','skipped','canceled')),
    attempt_count         int  NOT NULL DEFAULT 0,
    enqueued_at           timestamptz,
    sent_at               timestamptz,
    failure_reason        text,
    idempotency_key       uuid NOT NULL DEFAULT gen_random_uuid(),
    event_id              uuid REFERENCES nudge_event(id),  -- set on
success; the bridge
                                                     -- into the
reply/analytics spine
    meta                  jsonb NOT NULL DEFAULT '{}',      -- e.g. jit
schedule_key, payload
    created_at            timestamptz NOT NULL DEFAULT now(),
    updated_at            timestamptz NOT NULL DEFAULT now(),
    CHECK (template_id IS NOT NULL OR source = 'jit')       -- JIT content
comes from meta
);

-- Structural dedup: a template fires at most once per user per local day
CREATE UNIQUE INDEX uq_occurrence_routine
    ON nudge_occurrence (user_id, template_id, occurrence_date_local)
    WHERE template_id IS NOT NULL;

-- JIT dedup keys on the State Service token instead
CREATE UNIQUE INDEX uq_occurrence_jit
    ON nudge_occurrence (user_id, (meta->>'schedule_key'), occurrence_at)
    WHERE source = 'jit';
```

```
CREATE INDEX idx_occurrence_due
    ON nudge_occurrence (occurrence_at)
    WHERE status = 'pending';                          -- partial index for the
sweep

CREATE TABLE nudge_delivery_attempt (
    id               uuid PRIMARY KEY,
    occurrence_id    uuid NOT NULL REFERENCES nudge_occurrence(id),
    attempt_number   int NOT NULL,
    provider         text NOT NULL,                    --
'whatsapp_cloud','twilio'
    provider_msg_id  text,                             -- the WhatsApp message
id — this is
                                                       -- what
nudge_event.external_message_id
                                                       -- has been missing
    result           text NOT NULL CHECK (result IN
('success','transient_failure',
                                                       'permanent_failure')),
    error_detail     text,
    started_at       timestamptz NOT NULL,
    finished_at      timestamptz
);
```

Why a new table rather than extending `nudge_event`: `nudge_event` is a *fact* log — its consumers, including `_find_latest_event_id` in the reply matcher, treat "row exists" as "prompt was sent and may be awaiting a response." Pre-inserting `pending` rows would silently change that contract. It also hardcodes `Asia/Singapore` in a stored generated column and encodes the old 2-per-day cap in its unique index — both wrong for the multi-timezone goal. Plan (occurrence) and fact (event) stay separate; the `event_id` FK points from occurrence to event precisely so `nudge_event` needs **zero ALTERs** and the reply path cannot be destabilized.

6.3 Alterations to `nudge_schedule_config`

```
ALTER TABLE nudge_schedule_config
    ADD COLUMN preferred_time_local time NOT NULL DEFAULT '09:00',
    ADD CONSTRAINT chk_hour_aligned
        CHECK (EXTRACT(minute FROM preferred_time_local) = 0);
```

- `preferred_time_local` + the existing `timezone` column replace `preferred_time_utc`, which cannot express "9 AM local" across DST. Backfill from `preferred_time_utc` + `timezone`, dual-read for one release, then drop the old column.
- `enabled` becomes authoritative; the redundant `frequency = 'disabled'` state is normalized away by migration.
- `max_nudges_per_day` is now enforced at materialization (§7.1) — today it is selected and ignored.

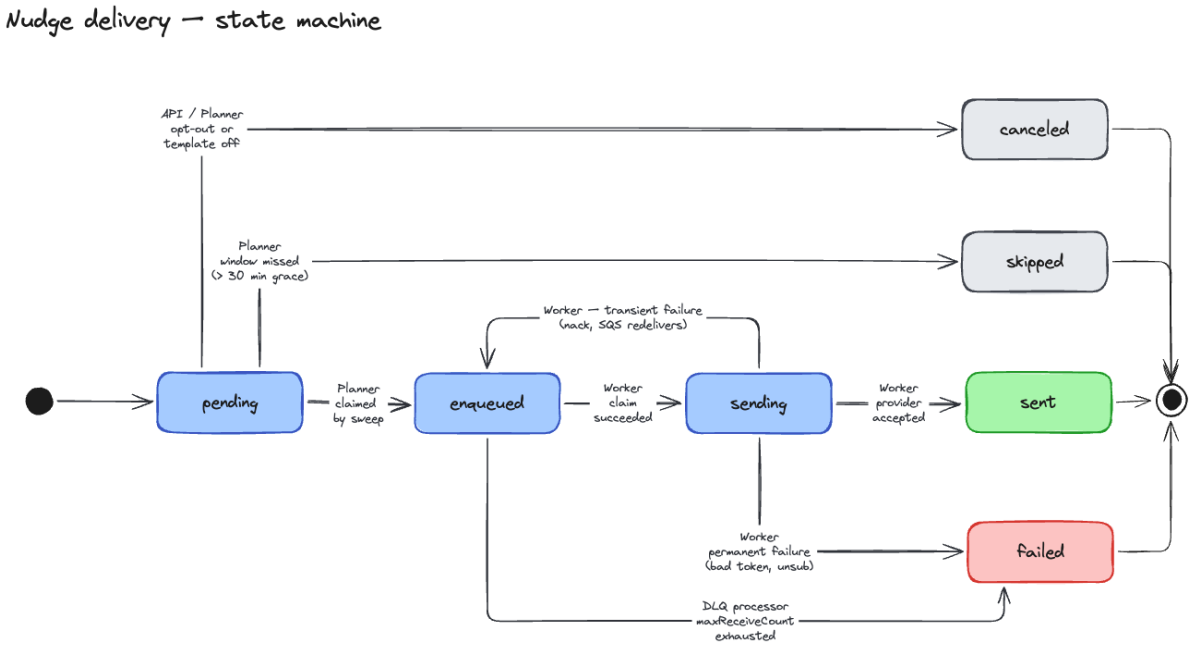
- **A config row is created for every user at signup** (and backfilled for existing users). This fixes the current silent exclusion where the `LEFT JOIN` + `WHERE` on config columns collapses to an inner join and users without a config row can never receive a nudge.

6.4 Extensions to nudge_template

```
ALTER TABLE nudge_template
    ADD COLUMN cadence_day   int,        -- day N of the 30-day programme;
NULL = not cadence-driven
    ADD COLUMN response_kind text CHECK (response_kind IN
('free_text','single_choice','yes_no')),
    ADD COLUMN choices       jsonb,
    ADD COLUMN score_map     jsonb,
    ADD COLUMN slot          smallint NOT NULL DEFAULT 1;
```

The Python registry's live entries (`mtip_01–03` on days 3/4/6, `pulse_02` day 14, `pulse_03` day 21) become Alembic **seed migrations** into this table — content changes stay code-reviewed but no longer require a worker redeploy. Planner selection, worker rendering, and reply-kind validation all read the table; once flipped, the registry module and `dep_nudge_template` are deleted. The commented-out cadence rows become `active = false` rows, visible in the database instead of buried in code comments.

6.5 Occurrence state machine



From	To	Actor	Trigger
pending	enqueued	Planner	Claimed by sweep
pending	canceled	API / Planner	User disables nudges, opts out of the channel, or the

From	To	Actor	Trigger
			template is deactivated
pending	skipped	Planner	Delivery window missed› beyond grace period (default 30 min)
enqueued	sending	Delivery worker	Message received, claim succeeded
sending	sent	Delivery worker	Provider accepted
sending	enqueued	Delivery worker	Transient failure (nack; SQS will redeliver)
sending	failed	Delivery worker	Permanent failure (bad token, unsubscribed)
enqueued	failed	DLQ processor	maxReceiveCount exhausted

Any transition whose `WHERE status = $expected` matches zero rows is treated as a duplicate/stale message and the SQS message is deleted without side effects. This is the core duplicate-suppression mechanism (G1, G6).

7. Scheduling design

7.1 Planner run (hourly via EventBridge cron, `cron(0 * * * ? *)`)

Each run performs two phases in order:

Phase A — Materialization. This replaces the daemon's per-tick Python filtering (`_get_eligible_users` + `resolve_templates_30_day`) with the same rules expressed as set-based SQL, run once per sweep:

1. **Eligibility:** `nudge_schedule_config.enabled` is true, at least one channel is enabled, and `day_number` — computed as `(today in the user's timezone) − user_profile.created_at::date` — is in 1..30. The `user_hash → user_id` resolution through `user_migration_map` happens here, once, so downstream stages carry a plain `user_id`.
2. **Insertion:** for each eligible user, insert one occurrence per active template with `cadence_day = day_number`, scheduled at the user's `preferred_time_local` converted to UTC for that date, `ON CONFLICT DO NOTHING` against `uq_occurrence_routine`. `max_nudges_per_day` is enforced here by capping inserts per user per `occurrence_date_local`.
3. The `nudge_test_phones` whitelist survives as a materialization predicate (config-driven), not a silent Python filter.

Two behaviors from the daemon are deliberately **not** carried forward: the ±60-minute "delivery window" match (exact hour-aligned occurrence times plus the grace period below replace it), and `check_nudge_sent_today` against `nudge_delivery_log` (the unique

index replaces it, which is what lets that table retire). Days with no cadence entry simply produce no rows — the current "No nudge content available" error path disappears.

Phase B — Claim and enqueue.

```
WITH claimed AS (
    SELECT id FROM nudge_occurrence
    WHERE status = 'pending'
      AND occurrence_at <= now()
      AND occurrence_at >  now() - interval '90 minutes'   -- grace period >
1 sweep interval
    ORDER BY occurrence_at
    FOR UPDATE SKIP LOCKED
    LIMIT 500                                              -- batch size,
tunable
)
UPDATE nudge_occurrence o
SET status = 'enqueued', enqueued_at = now(), updated_at = now()
FROM claimed WHERE o.id = claimed.id
RETURNING o.id, o.user_id, o.template_id, o.idempotency_key;
```

Rows are published to SQS **after** the transaction commits. If the process dies between commit and publish, a repair query in the next run re-publishes `enqueued` rows older than 10 minutes with no corresponding `sending`/`sent` progress (safe because the worker-side claim is conditional). Note that with an hourly cadence, this recovery path implies up to ~70 minutes of delay for the affected batch; the alarm on planner run failures (§13) is the primary defense.

The 90-minute grace period is deliberately longer than one sweep interval: if a single hourly run fails, the next run still picks up the previous hour's occurrences instead of skipping them. Occurrences older than the grace period transition to `skipped` rather than being delivered late — a 9 AM nudge arriving at noon is worse than no nudge.

`SKIP LOCKED` makes concurrent planner instances safe; the planner is nonetheless deployed as a single scheduled task since one instance comfortably handles the sweep (see §12).

7.2 Timezone requirements

- The user's IANA zone lives in `nudge_schedule_config.timezone` (updated by the API when the client reports a change); `preferred_time_local` + timezone are converted to UTC per date at materialization, so DST transitions resolve correctly.
- `nudge_occurrence.occurrence_date_local` is computed in the *user's* zone — this is the scheduling-side replacement for the `Asia/Singapore`-hardcoded `sent_date_sg`. (`nudge_event.sent_date_sg` keeps its current definition for analytics compatibility; whether to generalize it is tracked in §18.)
- A timezone or preferred-time change re-materializes that user's future `pending` occurrences (delete + re-insert within one transaction).

8. Queue design

8.1 Queues

Property	Main queue	DLQ
Type	Standard	Standard
Visibility timeout	90 s (≥ 3× worst-case send incl. provider timeout)	300 s
Long polling	20 s	—
maxReceiveCount	5 → redrive to DLQ	—
Retention	4 days	14 days
Encryption	SSE-SQS	SSE-SQS

8.2 Message contract (versioned)

```
{
  "schema_version": 1,
  "occurrence_id": "018f3a2e-...",
  "user_id": "018f39c1-...",
  "template_id": "018f38aa-...",
  "source": "routine",
  "idempotency_key": "7c9e6679-...",
  "enqueued_at": "2026-07-31T01:00:00Z",
  "trace_id": "abc123..."
}
```

The message is a **pointer, not a payload**: workers re-read the template content and the user's `nudge_schedule_config` from Postgres at send time, so template edits, deactivations, and opt-outs made after enqueue are honored. `idempotency_key` is passed to providers that support idempotent send semantics to suppress provider-side duplicates. For `source: "jit"` messages, `template_id` is null and the worker reads content from `nudge_occurrence.meta`.

9. Worker services

9.1 Delivery worker loop

1. Long-poll SQS (batch up to 10).
2. For each message: conditional transition `enqueued → sending`. Zero rows updated → delete message, continue (duplicate).
3. Load the template and the user's `nudge_schedule_config`. If nudges are disabled, the channel is opted out, or the template was deactivated → transition to `canceled/skipped`, delete message.
4. Send by **reusing the existing `whatsapp_api_service`** (`send_template_message` when a WhatsApp template name is set, else `send_checkin_template` — same branch logic as today, lifted out of the daemon). Record a `nudge_delivery_attempt` row, **capturing the provider message id from the send response**.
5. On success, in one transaction: occurrence `sending → sent`; insert `nudge_event` exactly as `log_nudge_event` does today (full body/choices/`response_kind` in `meta`,

`slot` from the template) **but with `external_message_id` set to the captured provider id**; mirror the text into `chat_memories` as an assistant message; set `nudge_occurrence.event_id`. Transient failure → `sending → enqueued`, do **not** delete (SQS redelivers). Permanent failure → `sending → failed`, delete message.

Populating `external_message_id` — passed as `None` at every current send site — is what finally activates the strict `context_id` branch of `_find_latest_event_id`, so swipe-replies attribute exactly instead of falling through to the 24-hour "latest unanswered event" heuristic. The reply-side code needs no change; it has been waiting for this value all along.

Channel scope (v1): WhatsApp only. The daemon's Telegram branch is unreachable (`telegram_chat_id` is never fetched into `user_data`) and the SMS branch calls a method that is commented out — today it raises `AttributeError`, gets swallowed by the per-channel `except`, and records a failed delivery. Neither branch is ported. SMS returns behind `sms_enabled` only when `_send_via_sms` is actually implemented; Telegram is dropped until there is a real requirement.

9.2 Scaling policy (lean by default, per G7)

- Metric: backlog per task = `ApproximateNumberOfMessagesVisible / running_task_count` (published as a custom metric).
- **Default min tasks: 0.** Traffic is bursty at the top of each hour, so the service scales to zero between sweeps. A scheduled scaling action sets desired count to 1 at :58 each hour, pre-warming a task so the burst never waits on Fargate cold start.
- Target tracking: 100 messages per task; max 20 tasks.
- **Capacity above the first task runs on Fargate Spot** via a capacity provider strategy (base 1 on-demand, remainder Spot). Spot interruption is safe here by construction: an interrupted task's unacked messages simply return to SQS and are redelivered.
- Scale-in protection during message processing via the ECS task scale-in protection API.

9.3 Planner

- Single scheduled Fargate task (EventBridge `cron(0 * * * ? *)` → ECS RunTask). RunTask is preferred over an always-on service: no idle cost between hourly runs, and a hung run cannot block the next one (each run is a fresh task with a 10-minute deadline — generous, since hourly batches are larger than minute batches).
- Overlap between runs is safe (§7.1), so no distributed lock is required.

10. API specification

All endpoints are served by the existing backend, under `/api/v1`. Auth: existing bearer-token middleware; all nudge routes are scoped to the authenticated user unless marked *admin*. Errors follow RFC 9457 (`application/problem+json`). All mutating endpoints accept an `Idempotency-Key` header.

10.1 User schedule preferences

The user-facing surface is the schedule config, not free-form nudge CRUD — the content programme is admin-managed through templates.

```
GET   /api/v1/notifications/nudge-config
PATCH /api/v1/notifications /nudge-config
```

```
{
  "enabled": true,
  "preferred_time_local": "09:00",
  "timezone": "Asia/Singapore",
  "channels": { "whatsapp": true, "sms": false },
  "max_nudges_per_day": 2
}
```

Validation: `preferred_time_local` must be **hour-aligned** to match the hourly sweep precision (D2) — clients should present hour-granularity pickers; `timezone` must be a valid IANA name; `max_nudges_per_day` in 0..10. A change to `preferred_time_local` or `timezone` re-materializes that user's future `pending` occurrences transactionally (§7.2). Pause/resume is `PATCH {"enabled": false}` / `{"enabled": true}` — no separate endpoints. Occurrences already `enqueued`/`sending` are not recalled, but the worker's config re-check (§9.1 step 3) means a disable still suppresses them at send time.

10.2 Template management (admin scope)

The existing `/nudge-templates` CRUD is **kept and becomes authoritative**, extended to accept the new columns:

```
GET    /api/v1/notifications/nudge-templates?active=true&cadence_day=3
POST   /api/v1/notifications/nudge-templates
PATCH  /api/v1/notifications/nudge-templates/{template_id}
DELETE /api/v1/notifications/nudge-templates/{template_id}     → active =
false (soft)
```

`PATCH`/`DELETE` accept `cadence_day`, `response_kind`, `choices`, `score_map`, `slot` alongside the existing fields. Deactivating a template cancels its future `pending` occurrences. Since the send path now reads this table (§6.4), these endpoints stop being the dead-end CRUD they are today — edits here change what users actually receive.

Test send (admin, synchronous, bypasses scheduler)

```
POST /api/v1/notifications/nudge-templates/{template_id}/test-send
```

Sends immediately to the caller's registered number (or a listed test phone) through the real delivery code path with a `test: true` flag; does not create an occurrence row. Rate-limited to 5/hour.

10.3 Occurrence history

```
GET /api/v1/notifications/nudge-occurrences?status=sent&from=...&to=...&cursor=...   # scoped to caller
GET /api/v1/notifications/nudge-occurrences/{occurrence_id}
```

The occurrence resource exposes `status`, `occurrence_at`, `template_id`, `sent_at`, `attempt_count`, `failure_reason`, and — on the detail route — embedded `nudge_delivery_attempt` rows and the linked `event_id`, so a support engineer can walk send → event → response in one place.

10.4 Upcoming programme preview

```
GET /api/v1/notifications/nudge-config/preview?count=10
```

Returns the caller's next N messages: materialized occurrences where they exist, computed beyond that from `cadence_day` templates vs the user's current day number. Lets clients render "what's coming" and makes the (currently code-buried) programme inspectable.

10.5 Admin / operational endpoints (admin scope)

```
GET  /api/v1/notifications/admin/occurrences?status=failed&from=...        # failure triage
POST /api/v1/notifications/admin/occurrences/{id}/replay                   # failed → pending (creates new attempt)
POST /api/v1/notifications/admin/dlq/redrive                               # SQS redrive DLQ → main queue
GET  /api/v1/notifications/admin/delivery/stats?window=1h                  # sent/failed/latency aggregates
```

`replay` requires the occurrence to be in `failed`; it resets to `pending` with a fresh `idempotency_key` so the planner re-claims it on the next sweep.

10.6 Internal contracts summary

Contract	Producer → Consumer	Medium	Versioning
Occurrence message (§8.2)	Planner → Delivery worker	SQS	`schema_version` field; workers reject unknown majors
Planner trigger	EventBridge → Planner	ECS RunTask input `{"run_type":"sweep"}`	Additive only
State transitions	Workers ↔ Postgres	SQL, conditional updates	Migration-gated

11. Reliability and failure modes

Failure	Behavior	Mechanism
EventBridge double-fire	No duplicates	`SKIP LOCKED` claim + conditional transitions
Planner dies after commit, before	Occurrence re-published ≤ 6 min later	Repair query (§7.1)

Failure	Behavior	Mechanism
publish		
SQS redelivers a processed message	Dropped silently	`enqueued → sending` claim matches 0 rows
Worker crashes mid-send	Redelivered after visibility timeout; provider idempotency key suppresses double-send where supported	Visibility timeout + idempotency key
Provider outage	Up to 5 transport retries with backoff (visibility timeout), then DLQ + alarm	maxReceiveCount, CloudWatch
Deploy of workers	Zero loss: unacked messages return to queue	SQS semantics; ECS drains with scale-in protection
RDS failover	Workers fail transitions, nack messages, retry after failover	Conditional updates are safe to repeat
User disables nudges after enqueue	Not sent	Worker re-checks config/template at step 3 (§9.1)

12. Performance and capacity requirements

- P1: Each claim batch must complete in < 5 s at 10 M pending occurrences (partial index, batch limit 500, loop until the due set is drained within the 10-minute task deadline).
- P2: End-to-end delivery latency (occurrence_at → provider accept) p95 < 5 min, p99 < 10 min under nominal load. Since sweeps fire at :00 and schedules are hour-aligned, this measures sweep start → provider accept.
- P3: Sustained throughput target: 1,000 sends/min with ≤ 5 delivery tasks; burst to 10,000 sends/min via auto-scaling within 3 minutes.
- P4: A single delivery task processes ≥ 200 msgs/min (10-msg batches, concurrent provider calls capped at 20 in-flight).

13. Observability

Metrics (CloudWatch, per environment): occurrences enqueued/sent/failed/skipped per minute; queue depth and age of oldest message; DLQ depth; delivery latency histogram (occurrence_at → sent_at); provider error rate by provider; planner run duration; repair-query re-publish count (should be ~0; sustained nonzero = bug).

Alarms: DLQ depth > 0 for 5 min (page); age of oldest main-queue message > 10 min (page — queue traffic is bursty at the top of each hour, and the burst should drain well within 10 min); planner run failure — a single missed hourly run (page, since the next retry opportunity is an hour away); delivery p95 latency > 10 min for one sweep cycle (ticket); repair re-publish count > 0 in two consecutive runs (ticket).

Tracing/logging: `trace_id` generated at enqueue, propagated through the message, worker logs, and `nudge_delivery_attempt`. Structured JSON logs to CloudWatch Logs; occurrence_id is the join key everywhere.

14. Security

- IAM: planner task role → `sqs:SendMessage` on main queue only; worker task role → `sqs:ReceiveMessage/DeleteMessage/ChangeMessageVisibility` on main queue, `sqs:SendMessage` on nothing; redrive permission held only by the admin API role.
- SQS server-side encryption; RDS encryption at rest; TLS everywhere in transit.
- Nudge content may contain user-generated text: treated as PII, excluded from logs (log IDs only), retained per existing data-retention policy; `nudge_delivery_attempt.error_detail` is scrubbed of provider payload echoes.
- Admin endpoints require the existing admin scope and are audit-logged.

15. Deployment and infrastructure

- One image, three entrypoints: `./app api`, `./app planner`, `./app worker` (G5/D4).
- **Build the worker image for ARM64 (Graviton)**: ~20% lower Fargate rates for identical work (G7). The API tier is unaffected.
- Three ECS task definitions; delivery worker as a Fargate service with the §9.2 scaling policy; planner via EventBridge → RunTask.
- **Networking (G7): do not provision a NAT gateway for this system.** Reuse the VPC's existing egress path if one exists; otherwise add SQS and ECR VPC interface endpoints, which are cheaper than NAT and remove per-GB processing fees for queue traffic.
- CloudWatch log groups for planner and workers get 30-day retention (never the default "forever").
- Infrastructure as code (Terraform): queues, DLQ + redrive policy, EventBridge rule, ECS services, alarms, IAM roles.
- CI/CD: existing pipeline builds the single image; deploy order is workers → planner → API (workers are backward-compatible with the previous message schema per §10.5).
- Config via SSM Parameter Store / env: batch size, grace period, horizon, visibility timeout, scaling targets.

16. Migration plan

The daemon currently runs three delivery paths (routine 09:00 SGT, doctor 7-day follow-ups at 18:00 SGT, hourly JIT). They migrate one at a time; the daemon retires only when all three are off it.

1. **Phase 0 — Schema.** Alembic migrations for `nudge_occurrence`, `nudge_delivery_attempt`, the `nudge_schedule_config` and `nudge_template` alterations (§6.3–6.4); seed `nudge_template` from the Python registry; backfill a config row for every existing user and `preferred_time_local` from `preferred_time_utc` + `timezone`; drop `dep_nudge_template`. **All runtime DDL is**

removed — `ensure_schema` running `routine_nudge_schema.sql` on the hot path is replaced by these migrations.

2. **Phase 1 — Shadow.** Planner + workers deploy in dry-run (log, don't send). Compare shadow decisions against the daemon's actual sends for ≥ 1 week — note the live programme is only 5 templates on days 3/4/6/14/21, so parity checking is small-N: validate day-number computation, template selection, and eligibility (especially users the LEFT-JOIN bug currently excludes, who will *gain* nudges at cutover — decide whether that's desired or backfilled configs should default `enabled = false`).
3. **Phase 2 — Cutover (routine).** Feature flag flips: workers send, the daemon's morning slot is disabled (flag read inside the send step, instant revert). Writes to `nudge_delivery_log` stop.
4. **Phase 3 — Cleanup (routine).** Remove the daemon's routine path, the dead `nudge_scheduler.py` CLI, the Python registry (template reads now hit the DB), the unreachable Telegram branch and the broken SMS branch, and drop `preferred_time_utc` after the dual-read release.
5. **Phase 4 — Migrate JIT.** `JitScheduleService.fire_due_schedules` is replaced by the planner materializing due `jit_schedules` rows into occurrences (`source = 'jit'`, `schedule_key` in `meta`, deduped by `uq_occurrence_jit`). The State Service contract is unchanged; only the delivery mechanics move.
6. **Phase 5 — Migrate doctor follow-ups.** `doctor_metrics_service` stops sending and becomes an occurrence *producer*: it inserts `source = 'doctor_followup'` rows anchored to the visit date. This is the template for any future nudge type — new producers write occurrences; nobody builds a new delivery path.
7. **Phase 6 — Retire the daemon.** Delete `DockerNudgeDaemon`, the `WORKER_ID == "0"` gating, and the import-time thread start in `app/main.py`. Drop `nudge_delivery_log` 90 days later.

Rollback at any phase = flip the relevant flag back; the state machine tolerates both systems observing (but only one sending, enforced by the flag being read inside the send step).

16.1 Deprecation schedule

Item	Replaced by	Stops	Removed
`nudge_delivery_log`	Occurrence state machine + `uq_occurrence_routine`	Phase 2 (writes)	Phase 6 + 90 days
`dep_nudge_template`	— (already dead)	—	Phase 0
Python cadence/registry (`cadence.py`, `registry.py`)	`nudge_template.cadence_day` + seed migrations	Phase 2	Phase 3
`nudge_scheduler.py` CLI	— (dead code, never imported)	—	Phase 3
`preferred_time_utc`	`preferred_time_local` + `timezone`	Phase 0 (dual-read)	Phase 3
`ensure_schema` runtime DDL	Alembic	Phase 0	Phase 0

Item	Replaced by	Stops	Removed
Daemon routine path	Planner + workers	Phase 2	Phase 3
Daemon JIT path	Planner (`source='jit'`)	Phase 4	Phase 6
Daemon doctor path	Occurrence producer	Phase 5	Phase 6
`DockerNudgeDaemon` + `WORKER_ID` gate	ECS Fargate services	—	Phase 6

16.2 Defects fixed by construction

Carried here explicitly so they're verified during shadow mode rather than rediscovered later: `external_message_id` never populated (breaks strict reply attribution — §9.1; this is the root cause your untracked `fix_nudge_response_misattribution.sql` patches around); LEFT JOIN + WHERE collapsing to an inner join and silently excluding config-less users (§6.3); `max_nudges_per_day` selected but never enforced (§7.1); SMS branch raising `AttributeError` into a swallowed except and recording phantom failures (§9.1); Telegram branch unreachable (§9.1); scheduling dates hardcoded to `Asia/Singapore` (§7.2); DDL on the hot path (§16 Phase 0); and the `slot` column hardcoded to 1 — superseded by per-template `slot`, still written to `nudge_event` for compatibility.

17. Cost analysis and lean-cost requirements

Assumptions: us-east-1, on-demand pricing as of July 2026 (Fargate $0.04048/vCPU-hr + $0.004445/GB-hr on x86, ~20% less on ARM64/Graviton; SQS standard $0.40/M requests, first 1M free; EventBridge Scheduler free below 14M invocations/month). The current daemon rides on existing EC2 capacity, so this architecture is nearly all net-new spend — the estimate shows it is small, and G7 requires it stays that way.

17.1 Estimate with lean defaults (scale-to-zero, Graviton)

Line item	Sizing	Monthly cost
Delivery workers	1× 0.25 vCPU / 0.5 GB ARM64, ~12 min/hour active (§9.2 pre-warm + burst drain)	~$1.50
Planner	730 runs × ~5 min, 0.25 vCPU / 0.5 GB ARM64	~$0.60
SQS (main + DLQ)	≤1 M nudges/mo ≈ 0.5–0.7 M requests w/ batching	$0 (free tier)
EventBridge Scheduler	730 invocations	$0 (free tier)
CloudWatch	1 custom metric, ~6 alarms, 30-day log retention	~$1–3
Total (existing VPC egress)		**~$3–5**
If no existing egress path	2× VPC interface endpoints (SQS, ECR)	~$15

For reference, running the delivery worker always-on instead of scale-to-zero costs ~$9/month (x86) — the scale-to-zero default in §9.2 is what keeps the floor near zero.

17.2 Marginal scaling costs

Each additional always-on-equivalent delivery task adds ~$7/month on Graviton; burst capacity above the first task runs on Fargate Spot at ~70% discount (§9.2), which is safe because SQS redelivers interrupted work. SQS adds roughly $0.15–0.40 per additional million nudges with batched send/receive/delete. At 10M nudges/month the entire pipeline stays under ~$30/month (G7), excluding channel-provider fees — which dominate total cost at any scale (e.g. SMS per-message rates) and should be modeled separately.

Cadence is not a cost lever: running the planner every minute instead of hourly changes total planner compute by roughly a dollar a month and stays inside the EventBridge free tier either way. The hourly choice is justified by operational simplicity and product fit, not cost.

17.3 Cost guardrails

- An AWS Budgets alert on the pipeline's cost-allocation tag at $25/month (notify) and $50/month (page). All resources in this system carry a `system:nudges` tag so spend is attributable.
- No NAT gateway may be provisioned for this system (§15). NAT is the single line item capable of tripling the bill.
- New cost-bearing resources (additional queues, provisioned concurrency, FIFO upgrades, extra environments) require a note in this document with an updated estimate.
- Re-validate this section when volume grows 10× or before adding a new delivery channel.

18. Open questions

1. **Identity consolidation:** `user_hash` (profile/config) vs `user_id` (events/sends) bridged by `user_migration_map` is the subsystem's deepest debt. This design confines the join to materialization, but should config migrate to `user_id` outright?
2. **Backfilled config defaults:** users currently excluded by the LEFT-JOIN bug will gain nudges once config rows exist. Default them `enabled = true` (intended behavior) or `false` (preserve observed behavior)?
3. **`user_metric_baseline` and `response_tags`:** declared, created by the schema file, never written. Implement writers (baseline computation in the planner? LLM tagging as an SQS consumer on replies?) or drop them?
4. **`nudge_event.sent_date_sg`:** keep the SG-hardcoded generated column for analytics compatibility, or add a per-user local date and migrate dashboards?
5. Do we keep `nudge_schedule_config.frequency` at all, given `enabled` + cadence templates now express everything it did?
6. Quiet hours / do-not-disturb windows — enforced at materialization or at send?

Copyrights @ HeyVenus Integrated Healthscience 2026

7. Should `skipped` occurrences notify the user or fail silently?
8. Multi-channel fallback (WhatsApp fails permanently → retry as SMS) — in scope once SMS is implemented?
9. Retention/archival for `nudge_delivery_attempt` and `nudge_occurrence` at scale (partition by month?).